

Nama: Ferdian Ilham Nugraha

Kelompok: Vanadium

1. Problem 1: The Last Astronaut (Pseudocode):

- https://github.com/vrthoz/Pemrograman_FerdianIlhamNugraha_Vanadium_TLS26.git**Pseudocode:**

```
PROGRAM TheLastAstronaut
INPUT N (jumlah astronot)
INPUT K (nilai eliminasi awal)

// Inisialisasi array untuk melacak status eliminasi
CREATE ARRAY eliminated OF BOOLEAN SIZE N+1
FOR i = 1 TO N DO
    eliminated[i] = FALSE
END FOR

remaining = N
current_pos = 1

PRINT "Urutan eliminasi: "

WHILE remaining > 1 DO
    count = 0

    // Proses pencarian astronot ke-K yang masih aktif
    WHILE count < K DO
        IF eliminated[current_pos] == FALSE THEN
            count = count + 1
            IF count == K THEN
                BREAK
            END IF
        END IF

        IF count < K THEN
            current_pos = current_pos + 1
            IF current_pos > N THEN
                current_pos = 1
            END IF
        END IF
    END WHILE

    // Eliminasi astronot terpilih
    eliminated[current_pos] = TRUE
    PRINT current_pos
```

```

// Modifikasi nilai K berdasarkan aturan
IF current_pos MOD 2 == 0 THEN
    K = K + 2
ELSE
    K = K - 1
END IF

IF K < 2 THEN
    K = 2
END IF

remaining = remaining - 1

// Tentukan posisi awal untuk hitungan berikutnya
IF remaining > 0 THEN
    REPEAT
        current_pos = current_pos + 1
        IF current_pos > N THEN
            current_pos = 1
        END IF
    UNTIL eliminated[current_pos] == FALSE
    END IF
END WHILE

// Cari astronot yang tersisa
FOR i = 1 TO N DO
    IF eliminated[i] == FALSE THEN
        PRINT "Astronot terakhir yang bertahan: ", i
        BREAK
    END IF
END FOR
END PROGRAM

```

- **Penjelasan Logika**

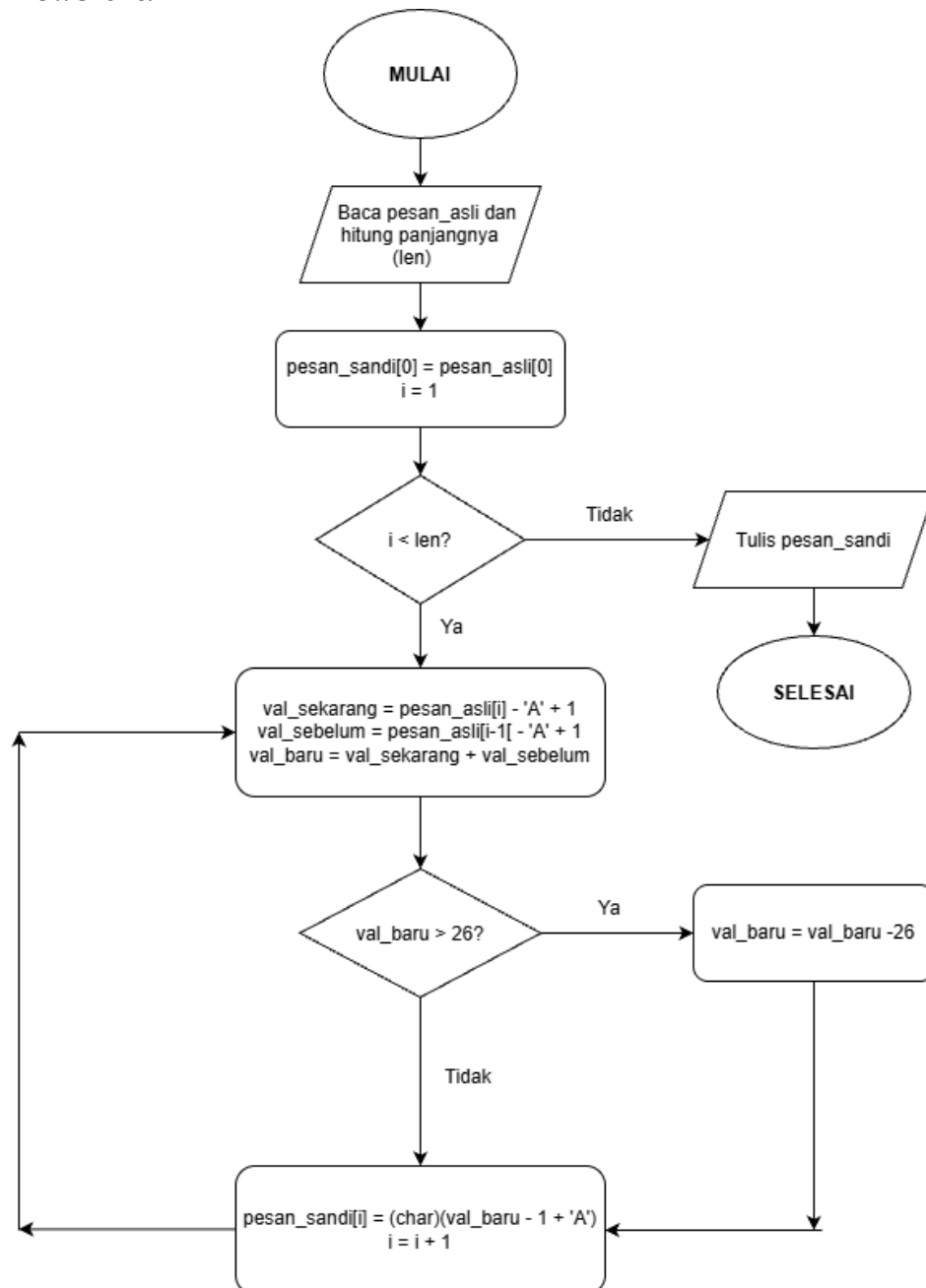
Logika penyelesaian masalah "The Last Astronaut" dirancang dengan menggunakan pendekatan simulasi array sirkular. Karena keterbatasan penggunaan fungsi pustaka, sistem pelacakan menggunakan array tipe boolean berukuran $N+1$ yang bertindak sebagai penanda (flag). Pada awalnya, seluruh indeks bernilai false, menandakan bahwa semua astronot masih aktif di dalam lingkaran. Saat iterasi berjalan, sebuah perulangan internal akan menghitung maju sebanyak K langkah, namun dengan secara cerdas mengabaikan indeks yang sudah ditandai true (dieliminasi). Ketika penghitung mencapai ujung array (melebihi N), indeks akan di-reset kembali ke 1. Hal ini mensimulasikan bentuk formasi lingkaran yang terus menyambung tanpa henti hingga jumlah astronot yang aktif (remaining) hanya tersisa satu.

Setelah seorang astronot berhasil diidentifikasi untuk dieliminasi, status pada array diubah menjadi true dan sistem langsung memproses aturan modifikasi nilai K .

Berdasarkan nomor urut astronot yang baru saja dieliminasi, program menerapkan logika kondisional: apabila nomor urut tersebut genap, nilai K akan ditambahkan 2, dan jika ganjil, K dikurangi 1. Mengingat adanya aturan batas minimum, fungsi pengecekan `IF K < 2 THEN K = 2` ditempatkan tepat setelah modifikasi untuk mencegah nilai K menjadi tidak valid atau memicu infinite loop pada perulangan berikutnya. Setelah parameter diperbarui, posisi `current_pos` digeser ke astronot aktif berikutnya yang ada di sebelah kanannya sebagai titik awal untuk siklus eliminasi selanjutnya.

2. Problem 2: Alien in The Middle (Flowchart):

- Flowchart:



- Penjelasan Logika:

Logika program "Alien-In-The-Middle" berpusat pada manipulasi tabel ASCII untuk merepresentasikan pergeseran (ciphering) indeks abjad tanpa bergantung pada library string bawaan. Program menyimpan input kata sebagai array of characters (C-string) dan memprosesnya karakter demi karakter. Mengingat karakter pertama tidak mengalami perubahan, sistem langsung menyalin indeks 0 dari pesan asli ke pesan sandi. Untuk karakter-karakter berikutnya, algoritma mengubah huruf kapital ke dalam format integer basis 1 hingga 26 dengan mengurangi nilai ASCII karakter tersebut dengan nilai ASCII huruf 'A', kemudian ditambah satu (Karakter - 'A' + 1). Konversi manual ini merupakan trik

matematis agar nilai huruf persis berkorespondensi dengan urutan alfabet murni (A=1, B=2, dst) sesuai aturan yang ditetapkan.

Hal terpenting dalam logika iterasi ini adalah pengambilan nilai referensi (huruf sebelumnya). Algoritma dirancang untuk selalu mengambil huruf referensi dari array pesan_asli, bukan dari huruf yang sudah dienkripsi, untuk memastikan pergeserannya valid dan sesuai dengan contoh ALIENS -> AMUNSG. Setelah nilai huruf saat ini dan nilai huruf asli sebelumnya dijumlahkan, program memvalidasi apakah angkanya melebihi 26 (batas maksimal alfabet Z). Jika melewati batas, nilai tersebut dikurangi 26 berulang kali (siklus wrap-around pengganti modulo) agar tetap kembali ke dalam rentang 1-26. Angka akhir tersebut kemudian direstorasi kembali menjadi tipe data character ASCII yang valid dan dimasukkan ke dalam array hasil.